



भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU



NLP Foundations

Terminologies & Text Processing Importance

By
Parmveer Nandal

Objective:

To understand fundamental concepts in text processing and their necessity for building robust NLP models

Why Do We Need Text Pre-processing?

Why Do We Need Text Pre-processing?

Text pre-processing is crucial for preparing data before feeding it to a model

1. Improve Data Quality

- Removes irrelevant elements like HTML tags, stop words, punctuation, and extra whitespace
- Ensures the data is clean and can capture proper, meaningful information, which is vital for effective model training

Why Do We Need Text Pre-processing?

2. Reduce Dimensionality

- Simplifies data by removing redundant forms (e.g., converting "running" to "run" by removing prepositions or variations).
- Decreases the size of the data, leading to faster processing and saving computational resources

3. Enhance Model Accuracy

- Clean and well-processed data directly contributes to higher model accuracy.
- It also improves the speed at which the model operates.
- If the data is good, the answer will be good

When to Perform Text Pre-processing?

When to Perform Text Pre-processing?

- Before Model Training
- During Feature Extraction, to identify important and relevant features from the data

Terminologies in NLP

Corpus: A large collection of text documents.

- It is the entire dataset that will be analyzed and used for training NLP models.
- It can consist of multiple pages, books, or any extensive collection of text.
- Example: A collection of all articles from a newspaper archive

Corpora is the plural of corpus, referring to a collection of corpuses, though *corpus* is more commonly used

Terminologies in NLP

Document: A single piece of text within a corpus

- It can be an **article, a paragraph, or even a single sentence**, depending on how the data is structured
- *Example:* In a corpus of books, each book could be a document, or even each chapter within a book could be considered a document

Terminologies in NLP

Vocabulary: The set of all unique words present in the corpus

- It acts as the "dictionary" of terms that the NLP model will work with.
- Only unique words are counted in the vocabulary; repeated words are not counted multiple times.
- Example: For the sentences "I am a Prince" and "I am a Mentor", the vocabulary would be {I, am, a, Prince, Mentor}

Terminologies in NLP

Tokens: The smallest, meaningful units into which a sentence or text is divided

- These are the basic building blocks derived from raw text
- A token can be:
 - A **word** (e.g., "Prince")
 - A **single character**
 - Even a **sentence** itself, depending on the level of tokenization
- *Example:* For the sentence "I am a Prince", the tokens would be "I", "am", "a", and "Prince"

Terminologies in NLP

Frequency: The number of times a particular token or word appears in a document

- It measures the occurrence count of a specific term
- Example: If the word "Rahul" appears 5 times in a document, its frequency is 5

Terminologies in NLP

Gram: In NLP, "gram" refers to a "group"

- Specifically, it denotes a group of words
- Purpose: Grams help in capturing the meaning of text by considering sequences of words, as individual words might not convey full context

Terminologies in NLP

Types of Grams:

- **Uni-gram: A group containing one word**
 - Example: From "I love NLP", the uni-grams are "I", "love", "NLP".
- **Bi-gram: A group containing two consecutive words.**
 - Example: From "I love NLP", the bi-grams are "I love", "love NLP".
- **Tri-gram: A group containing three consecutive words.**
 - Example: From "I love NLP", the tri-gram is "I love NLP".



भारतीय प्रौद्योगिकी
संस्थान जम्मू
INDIAN INSTITUTE OF
TECHNOLOGY JAMMU



NLP Foundations

Text Cleaning and its Sequence in NLP Pre-processing

Objective:

To understand the importance of text cleaning, its practical application (punctuation, white space removal), and its place within the overall NLP pre-processing pipeline

Why Do We Need Text Cleaning

Text cleaning is the first and most crucial step in text pre-processing

4. Prepares for Next Steps

It is essential to clean text *before* converting it into tokens or other representations. Otherwise, the tokens would contain irrelevant characters (like punctuation or special characters), corrupting the meaning

Sequence of Text Pre-processing Steps

A systematic order is followed to prepare text data for NLP models

1. **Text Cleaning:** The initial step, involving the removal of:

- Punctuation
- White Spaces
- Numbers
- Special Characters

2. **Tokenization:** Breaking down text into smaller units (words, sentences, or characters)

- Types: Word Tokenization, Sentence Tokenization, Character Tokenization

Sequence of Text Pre-processing Steps

- **Stop Words Removal:** Eliminating common words (e.g., "the", "is", "and") that add little meaning
- **Stemming & Lemmatization:** Reducing words to their root form
- **Text Segmentation:** Breaking text into logical units like topics, sentences, or chunks
- **N-grams:** Creating sequences of 'N' words (optional).
 - **Types: Uni-gram, Bi-gram, Tri-gram**

Sequence of Text Pre-processing Steps

- **Vectorization: Converting text into numerical representations.**
 - Techniques: One-Hot Encoding, TF-IDF, Bag of Words
- **Word Embedding:** Advanced techniques to represent words as dense vectors, capturing semantic meaning
 - **Techniques: Word2Vec, FastText**

Text Cleaning: Punctuation Removal

What are Punctuations?

- Characters like periods (.), commas (,), exclamation marks (!), question marks (?), etc.

- **Why Remove Them?**

- Punctuations are noise for training NLP models.
- Their presence can decrease the accuracy of the model's predictions.
- They are irrelevant to the core meaning the model needs to learn

Text Cleaning: White Space Removal

What is White Space Removal?

- The process of cleaning up excessive or unnecessary spaces within a string or text
- This includes removing leading/trailing spaces, multiple spaces between words, and newlines
- **Why Remove Them?**
 - To ensure the string is properly formatted and clean
 - Reduces inconsistencies in data that could affect processing.

Text Cleaning is Foundational:

Handling punctuation and white spaces is the **very first and most critical step** in preparing text data

Impact on Tokenization:

If text is not cleaned first, subsequent tokenization will result in "falsy" (unwanted) tokens containing punctuations or special characters, which can **corrupt the meaning** and quality of your model's input

Understanding Text Tokenization

What is Tokenization?

- Tokenization is the process of splitting text into smaller pieces called tokens.
 - These tokens can be words, numbers, or punctuation marks.
 - For example, if "We love NLP!" is tokenized, it can become "We", "love", "NLP", and "!" as separate tokens
-
- **Why Remove Them?**
 - To ensure the string is properly formatted and clean
 - Reduces inconsistencies in data that could affect processing.

Understanding Text Tokenization

Why Tokenize (Purpose)?

- The primary purpose is to convert text into a numerical (vector) form
- It's not possible to convert an entire sentence directly into numerical vectors
- Tokens create manageable units that can then be converted into vectors

When to Tokenize?

- Tokenization is the second stage in text processing, performed after data cleaning
- This ensures the word's actual meaning is preserved and not distorted

Advantages of Tokenization

Creates Manageable Units:

- Allows text to be broken down into smaller, manageable units that can be converted into vectors

Facilitates Part of Speech (POS) Tagging:

- Helps in identifying the grammatical role of each word (e.g., noun, pronoun) within a sentence, which is difficult with a whole sentence. For instance, "He" can be tagged as a 'pronoun'

Advantages of Tokenization

Aids in Named Entity Recognition (NER):

- Enables the identification of specific entities like names within the text

Context Extraction:

- Can be used to help extract context from the text

Disadvantages of Tokenization

Loss of Context:

- When a sentence is broken into individual pieces (tokens), its original context can be lost

Ineffectiveness in Multi-Language Texts:

- Tokenization can be ineffective or pose significant challenges when dealing with multiple languages

Types of Tokenization

Tokenization is not limited to converting text into single words; there are many types

Key Types:

- **Word Tokenization:** Splitting text into individual words3... This is generally the most important and frequently used type
- **Sentence Tokenization:** Splitting text into individual sentences. This is done based on delimiters like spaces, commas, or explanation marks
- **Character Tokenization:** Splitting text into individual characters

Types of Tokenization

Tokenization is not limited to converting text into single words; there are many types

Other Mentioned Types:

- Sub-word Tokenization
- White Space Tokenization: Splitting based on spaces

Stop Words:

Filtering Unimportant Text Units

What are Stop Words?

- Stop words are irrelevant words or words whose meaning is not important when considered as a single unit.
- They do not carry significant meaning on their own

What are Stop Words?

- **Purpose:** To remove unnecessary words from text
- **Examples:** Words like "a", "me", "to", "all", "is", "an", "of", "as", "p", "not", "no", "never". Stop words are defined differently for each language

Why Remove Stop Words? (Advantages)

- **Dimensionality Reduction:** Removing stop words reduces the number of tokens and the dimensionality when converting text into vectors.
 - For instance, if you have 1000 tokens and 200 are stop words, you are left with 800 meaningful tokens, reducing the vector's dimension
- **Improved Contextual Meaning:** The remaining tokens will all have meaning, leading to a **better capture of the actual meaning** and context when creating vectors

Disadvantages of Removing Stop Words

- **Loss of Relation/Context:** In certain tasks like sentiment analysis, removing stop words can be problematic because negative words (e.g., "not," "no," "never") are often removed, which can distort the sentiment. The relationship between words might be lost if articles or prepositions are removed
- **Multi-Language Complexity:** Removing stop words can be challenging when dealing with texts in different languages

Disadvantages of Removing Stop Words

- **Loss of Relation/Context:** In certain tasks like sentiment analysis, removing stop words can be problematic because negative words (e.g., "not," "no," "never") are often removed, which can distort the sentiment. The relationship between words might be lost if articles or prepositions are removed
- **Multi-Language Complexity:** Removing stop words can be challenging when dealing with texts in different languages

Stemming & Lemmatization: Normalizing Word Forms

Why Normalize Word Forms?

- **Purpose:** To convert various forms of a word (e.g., "say," "says," "said") into a single root word

Why Normalize Word Forms?

Goal:

- Reduce the number of unique tokens.
- Decrease dimensionality when converting text to numerical vectors.
- Prevent model confusion and speed up training by reducing redundant features.
- Ensure different word forms of the same base word map to a single representation, making vector conversion more efficient and meaningful

Stemming

Definition:

- A process that cuts off prefixes or suffixes from a word to get a base form, often called a "stem".
- Key Characteristic: No guarantee that the resulting stem will be an actual, meaningful root word. It's like "chopping" the word

Stemming

Advantages:

- **Simplicity:** It's a relatively simple technique
- **Speed:** Generally faster because it doesn't consider meaning or part-of-speech (POS)

Disadvantages:

- **Non-meaningful Words:** Can produce non-meaningful or incorrect words as output
- **Loss of Meaning/Context:** The original meaning or context of the word might be lost

Stemming

When to Use:

- When fast processing is prioritized, and the precise meaning or POS of the word is not crucial

Example (NLTK PorterStemmer):

- "running" → "run" (Correct)
- "easily" → "easili" (Incorrect/Non-meaningful)

Lemmatization

Definition:

- The process of converting a word to its actual root word (lemma), ensuring the resulting word is meaningful and grammatically correct. It uses dictionary-related words
- Key Characteristic: Provides a guarantee for a meaningful word

Lemmatization

Advantages:

- **Meaningful Output:** Always results in a meaningful and contextually relevant word
- **Better Context Preservation:** Helps in maintaining the word's meaning and extracting context

Disadvantages:

- **Slower:** Generally slower than stemming because it involves more complex linguistic analysis (like dictionary lookups and POS tagging)

Lemmatization

When to Use:

- When semantic accuracy and the contextual meaning of words are critical for the task (e.g., sentiment analysis, where negative words like "not" shouldn't be lost)

Example (NLTK WordNetLemmatizer):

- "saying" → "say" (Correct)
- "took" → "take" (Correct)
- "easily" → "easily" (Correct)

Lemmatization vs Stemming

1. Meaningful Word Guarantee:

- Lemmatization: Yes, guarantees a meaningful root word
- Stemming: No, does not guarantee a meaningful root word

2. Speed:

- Stemming: Faster
- Lemmatization: Slower

3. POS Consideration:

- Lemmatization: Considers Part-of-Speech (POS).
- Stemming: Does not consider POS

Lemmatization vs Stemming

1. Meaningful Word Guarantee:

- Lemmatization: Yes, guarantees a meaningful root word
- Stemming: No, does not guarantee a meaningful root word

2. Speed:

- Stemming: Faster
- Lemmatization: Slower

3. POS Consideration:

- Lemmatization: Considers Part-of-Speech (POS).
- Stemming: Does not consider POS

Text Normalization & Text Segmentation

“Preparing Text for Numerical Conversion”

Text Normalization

Definition:

The process of converting text to a "normal" or standardized form

Primary Purpose (Case Normalization):

To handle words that are the same but appear in different cases (e.g., "Word" vs. "word").

- If not normalized, "Word" and "word" would be treated as two separate words/tokens
- This increases dimensionality and the number of tokens, which is undesirable for converting text into vectors or numerical formats
- The goal is to reduce these numbers for more efficient processing

Text Normalization

How to Normalize (Lowercasing Example):

- The most common method is converting all text to lowercase
- This can be done easily using the `.lower()` function available in Python for strings
- NLTK also provides ways to perform this, but it essentially relies on the same basic string lowercasing functionality

Text Segmentation

Definition:

- The process of splitting a text into different parts or "segments"
- These segments can be smaller words, sentences, or groups of words

Relationship to Tokenization:

- Text segmentation serves a similar purpose to tokenization, where text is broken into smaller, manageable units

Text Segmentation

How to Segment (Using NLTK):

The NLTK (Natural Language Toolkit) library is commonly used

- **Sentence Segmentation:** Use `sent_tokenize()` to split a paragraph or longer text into individual sentences. Each sentence becomes a separate segment
- **Word Segmentation:** Use `word_tokenize()` to split sentences or text into individual words. Each word becomes a separate token
- **Process:** Import the necessary functions (e.g., `sent_tokenize`, `word_tokenize`), ensure NLTK data is downloaded (if required), and then pass the text to the respective function. The output is typically a list of segmented units

POS Tagging & NER:

Understanding Text Structure

Why Analyze Text Structure?

- The primary goal is to convert text into numerical vectors
- Providing more detailed information about words or tokens helps in better meaning and context capture
- This leads to clearer context and better-formed vectors

Part-of-Speech (POS) Tagging

Definition: The process of **tagging words with their grammatical role** or "Part of Speech" (e.g., noun, verb, adjective, determiner)

Purpose:

- Helps to understand what kind of word it is.
- Enhances the meaning capture and contextual understanding of the text
- Aids in **parsing** text, which is useful for tasks like web scraping

When to Use: It is applied after tokenization

Part-of-Speech (POS) Tagging

Advantages:

- Provides more contextual information about tokens, allowing for categorization.

Disadvantages:

- Can be language dependent and may have contextual limitations based on the language

Part-of-Speech (POS) Tagging

Examples of Tags:

- "The" → Determiner (DT)
 - "Fox" → Noun (NN)
 - "Lazy" → Adjective (JJ)
 - "Jumps" → Verb (VBD)
 - Specific notations are used, e.g., NN for Noun, JJ for Adjective
-
- Apply **`nltk.pos_tag()`** to the list of tokens
 - The output provides each token paired with its POS tag

Named Entity Recognition (NER)

Definition:

- The process of identifying and categorizing "named entities" within text, such as persons, places, or organizations

Relationship to POS Tagging:

- It's considered a **next or similar step** to POS tagging, serving the same core purpose of adding detailed information about words. A word can be part of its POS or an NER

Named Entity Recognition (NER)

Purpose:

- Provides more detailed information about words to help with context capture before vector creation

When to Use:

- Applied after tokenization and typically after POS tagging (as shown in the implementation example)

Named Entity Recognition (NER)

Examples of Entities:

- "Apple" (as in the company) → Proper Noun (NNP)
 - "UK" → Noun (NNP) or Geo-Political Entity (GPE)
-
- Tokenize the sentence
 - Perform POS tagging on the tokens
 - Apply **`nltk.ne_chunk()`** to the POS-tagged tokens
 - The output will identify and group named entities

One-Hot Encoding:

A Foundation for Word Embeddings

“Converting Text to Numerical Format”

Introduction to Word Embeddings

Definition:

- The process of converting text into numerical formats (often called vectors)

Importance:

- Crucial for Generative AI and Vector Databases
- Machine Learning models understand numerical values, not raw text

Introduction to Word Embeddings

Techniques:

- Various methods exist for word embedding, including:
 - One-Hot Encoding
 - Bag-of-Words
 - TF-IDF (Term Frequency-Inverse Document Frequency)
 - Word2Vec
 - GloVe

Introduction to Word Embeddings

Evolution of Techniques:

- Newer techniques emerged to solve the problems inherent in older ones.
- One-Hot Encoding was the first method, and its issues led to the development of Bag-of-Words, TF-IDF, Word2Vec, and GloVe in sequence

One-Hot Encoding

Definition:

- A binary-based encoding method where only one value is "hot" and all others are "cold" (0).

Purpose:

- The simplest and most basic way to convert textual data into a numerical format for models

One-Hot Encoding

How it Works:

- Each unique word in the vocabulary is assigned a unique index
- A vector is created for each word, where the position corresponding to that word's index is **set to 1**, and all other positions are **set to 0**
- This converts a text line into a numerical representation

One-Hot Encoding

Examples:

- For the sentence "I love machine learning":
- "I" $\rightarrow$ 1 (if "I" is the first word in the ordered vocabulary)
- "love" $\rightarrow$ 1.
- "machine" $\rightarrow$ 1.
- "learning" $\rightarrow$ 1.

For colors "Red Blue Black":

- "Red" $\rightarrow$ 1.
- "Blue" $\rightarrow$ 1.
- "Black" $\rightarrow$ 1.

One-Hot Encoding

- **Implementation:** Can be done using basic Python code, Pandas, NumPy, or more commonly and efficiently using `sklearn.preprocessing.OneHotEncoder`

One-Hot Encoding

Advantages of One-Hot Encoding

- **Simplicity:** It is a very simple and basic technique to understand and implement
- **Clear Mapping:** Provides a clear and direct mapping between words and their unique numerical representations

One-Hot Encoding

Disadvantages/Limitations of One-Hot Encoding

- **High Dimensionality:** As the vocabulary size increases, the generated vectors become extremely long, leading to a high number of dimensions (features). For example, if there are 10,000 unique words, each word's vector will be 10,000 elements long
- **Sparsity:** Most of the elements in the vectors are zeros, resulting in sparse matrices. This means a lot of storage space is used for redundant information (zeros)

One-Hot Encoding

Disadvantages/Limitations of One-Hot Encoding

- **No Semantic Meaning/Context:** One-Hot Encoding fails to capture any relationship or contextual similarity between words. For instance, "king" and "queen" or "cat" and "kitten" would be equally distant, even though they are semantically related. There is "no semantic meaning" or "context" captured between words
- **Out-of-Vocabulary (OOV) Issue:** It cannot handle new words that were not present in the original vocabulary used to create the encoding. If a new word appears, it has no representation

One-Hot Encoding

Disadvantages/Limitations of One-Hot Encoding

- **Inflexibility (Cannot Add/Modify):** Once the encoding is established based on a fixed vocabulary, it's difficult to add new words or modify the structure without re-creating the entire encoding

One-Hot Encoding

Conclusion

- Due to its significant disadvantages, especially concerning dimensionality, sparsity, and lack of semantic meaning, One-Hot Encoding is generally not preferred for complex NLP tasks.
- However, it serves as a fundamental first step in understanding how text can be converted to numerical data for machine learning models

One-Hot Encoding

Conclusion

- Due to its significant disadvantages, especially concerning dimensionality, sparsity, and lack of semantic meaning, One-Hot Encoding is generally not preferred for complex NLP tasks.
- However, it serves as a fundamental first step in understanding how text can be converted to numerical data for machine learning models

Bag of Words (BoW): Improving Text Representation

“A Step Towards Better Word Embeddings”

What is Bag of Words (BoW)?

Definition:

- Bag of Words is a technique to convert textual data into a numerical format (vectors)

Purpose:

- It was developed as a new technique to address the problems of One-Hot Encoding. The name "Bag of Words" suggests collecting words and potentially involves counting.
- It involves creating a "bag" (vocabulary) of all unique words and then counting how many times each word appears in a given document

Core Idea:

- It creates a numerical representation based on the frequency of words

Steps to Create a Bag of Words Model

1. **Corpus to Documents:** Convert the entire corpus into individual documents.
2. **Tokenization:** Tokenize the documents (separate words).
3. **Vocabulary Creation:** Build a vocabulary of unique words from all documents.
Each word appears only once in the vocabulary.
4. **Frequency Counting:** Count the frequency of each word from the vocabulary within each document
5. **Arrangement/Encoding:** Arrange these counts into a vector for each document, where the position corresponds to a word in the vocabulary, and the value is its frequency in that document

Example Walkthrough

Corpus Example:

- Document 1 (d1): "I love machine learning"
- Document 2 (d2): "machine learning is fun"

Vocabulary:

- "I", "love", "machine", "learning", "is", "fun"
- (Each unique word from the corpus).

Example Walkthrough

Encoding based on Frequency

For d1 ("I love machine learning"):

"I": 1

"love": 1

"machine": 1

"learning": 1

"is": 0

"fun": 0

Resulting vector: _ _ _ _ _

For d2 ("machine learning is fun"):

"I": 0

"love": 0

"machine": 1

"learning": 1

"is": 1

"fun": 1

Resulting vector: _ _ _ _ _

Example Walkthrough

Key Insight:

If a word appears multiple times in a document (e.g., "love" twice), its count in the vector would increase accordingly

Properties of Bag of Words

- **Unordered:** The order of words in the original document is not preserved; it's a "bag" of words, not a sequence
- **Frequency-Based:** The representation relies solely on the frequency (count) of words

Bag of Words – Use Cases

- **Text Representation:** Used when converting numbers to vectors for various tasks
- **Classification Problems:** Useful when the problem involves classifying text
- **Small Datasets:** Typically more effective for smaller datasets
- **Feature Extraction:** Can be used for feature extraction

Bag of Words – Advantages

- **Simplicity:** It is a simple technique to understand and implement
- **Solves Out-of-Vocabulary (OOV) Issue (Partially):** A key advantage is that it can handle new occurrences of words (even if they were not in a document previously, they will be in the vocabulary). If a word appears multiple times, its count can simply be increased, effectively allowing for addition or modification of counts, which One-Hot Encoding struggled with

Bag of Words – Disadvantages/Limitations

- **High Dimensionality:** While slightly improved from One-Hot Encoding, it still suffers from high dimensionality as the vocabulary size grows
- **Sparsity:** Many values in the vectors will still be zeros, leading to sparse matrices
- **No Semantic Meaning/Context:** It fails to capture any semantic meaning or context between words. It only tells you if a word is present and how many times, but not its relationship to other words
- **Order Not Maintained:** As noted, it does not consider word order, losing valuable sequential information that is crucial for understanding sentence meaning

TF-IDF:

Unveiling Word Importance in Documents

“The Third Step in Word Embeddings”

Introduction to TF-IDF

- **Definition:** TF-IDF stands for Term Frequency-Inverse Document Frequency
- **Purpose:** Developed as the third technique in word embeddings (following One-Hot Encoding and Bag of Words). Its primary goal is to determine the importance of a word within a specific document, relative to a collection of documents (corpus)

Introduction to TF-IDF

Core Idea:

- It combines two metrics to assign a weight to each word, reflecting its relevance

Components of TF-IDF

Term Frequency (TF)

- **Definition:** Measures how frequently a word appears within a particular document. It indicates how "common" a word is in that specific document.
- **Formula:** $TF(\text{word}, \text{document}) = \frac{\text{Number of times 'word' appears in 'document'}}{\text{Total number of words in 'document'}}$

Components of TF-IDF

Term Frequency (TF)

- **Example:** For the word "machine" in the document "I love machine learning" (4 words total), if "machine" appears once, $TF = 1/4 = 0.25$

Components of TF-IDF

Inverse Document Frequency (IDF)

- **Definition:** Measures how rare or unique a word is across all documents in the corpus. It assesses how many documents contain that specific word
- **Purpose:** Identifies how "rare" a word is. A word that appears in fewer documents is considered more unique and thus more important

Components of TF-IDF

Inverse Document Frequency (IDF)

- **Formula:** $\text{IDF}(\text{word}) = \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing 'word'}} \right)$
- **Example:** For the word "machine" in a corpus of 3 documents where "machine" appears in 2 of them, $\text{IDF} = \log(3/2)$

TF-IDF Calculation

- **Combined Formula:** The TF-IDF score for a word in a document is the product of its TF and IDF scores: $\text{TF-IDF}(\text{word}, \text{document}) = \text{TF}(\text{word}, \text{document}) * \text{IDF}(\text{word})$
- **Result:** The TF-IDF score provides the "Importance of Word"

Interpreting TF-IDF Scores

- **High TF-IDF Score:**

- Indicates that the word is highly important in that specific document
- Suggests the word is relatively unique to that document
- If a word consistently has a high TF-IDF score across multiple documents, it can be considered important for the entire corpus

- **Low TF-IDF Score:**

- Indicates that the word is common across many documents and thus less significant/unique

Real-world Application:

- Google uses this concept to determine the most relevant webpages to display for a search query, prioritizing pages with higher TF-IDF scores for the query words. A word is important if it is unique; if TF-IDF is high, the word is unique and important

Advantages of TF-IDF

- **Highlights Unique/Important Words:** Effectively identifies words that are significant to a specific document and unique within the corpus
- **Useful for Text Classification:** Employed in text classification problems
- **Information Retrieval:** Valuable in information retrieval systems (e.g., search engines)

Disadvantages/Limitations of TF-IDF

- **No Order/Grammar Maintained:** Like previous methods, TF-IDF does not account for word order or grammatical structure, leading to a loss of semantic context
- **No Semantic Meaning/Context:** It fails to capture the relationship or contextual similarity between words (e.g., "king" and "queen" are treated as distinct entities, even if semantically related)

Disadvantages/Limitations of TF-IDF

- **Sparsity:** Can still result in sparse matrices (many zero values) when dealing with large vocabularies
- **High Dimensionality:** As the vocabulary grows, the resulting vectors can still have high dimensionality

Word2Vec: Revolutionizing Word Embeddings

“Moving Beyond Traditional Text Representation”

What is Word2Vec?

- **Definition:** Word2Vec is a concept and a technique to convert words (textual data) into numerical vectors
- **Purpose:** It is a part of the broader concept known as Word Embedding. The name "Word2Vec" literally means converting words to vectors
- **Core Idea:** It transforms textual words into numerical vector representations, aiming to capture semantic meaning and relationships between words

The Evolution of Word Embeddings:

Why Word2Vec?

Previous techniques like One-Hot Encoding, Bag of Words (BoW), and TF-IDF struggled with key problems:

- Lack of Semantic Meaning: They could not capture the contextual or relational meaning between words
- High Dimensionality: Resulted in very large, sparse vectors, especially with a growing vocabulary
- Lack of Generalization: Difficulty in generalizing to new words or contexts

The Evolution of Word Embeddings:

Why Word2Vec?

Word2Vec provides solutions to these challenges, specifically aiming to capture semantic meaning and address high dimensionality

The Core Concept: Semantic Meaning through Vector Space

How it Works: Word2Vec posits that if we convert text into vectors, we can capture semantic meaning

The King-Man + Queen-Woman Analogy:

- Consider the equation: $\text{King} - \text{Man} + \text{Woman} = \text{Queen}$
- This implies that the relationship between King and Queen is similar to the relationship between Man and Woman

Key Examples and Use Cases

1. Semantic Relationships (Analogies)

Word2Vec can model complex semantic and syntactic relationships:

- Gender & Royalty
 - "King" - "Man" + "Woman" $\approx$ "Queen"
- Country–Capital Relationship
 - "Paris" - "France" + "Germany" $\approx$ "Berlin"
- Grammatical / Syntactic Relationships
 - "Bigger" - "Big" + "Small" $\approx$ "Smaller"

Key Examples and Use Cases

2. Finding Similar Words

A trained Word2Vec model can retrieve words with similar meanings or contexts:

- For "dirty" → "smelly", "filthy"
- For "running" → "jogging", "ran", "runs"

This is useful for:

- Synonym detection
- Query expansion
- Text similarity tasks

Key Examples and Use Cases

3. Identifying Unrelated Words (Odd-One-Out)

Word2Vec can identify the word that does not belong in a group:

- Given: ['fire', 'water', 'land', 'sea', 'air', 'car']
- Output: 'car' (since the others are natural elements)

The Core Concept: Semantic Meaning through Vector Space

Cosine Similarity:

- This idea is based on **cosine similarity**, which measures the distance (or similarity) between vectors when plotted
- If words are **semantically similar**, their vector representations will be **closer together** in the vector space, meaning their cosine distance will be less.
- Example: When plotted, "Man" and "Woman" would be near each other, and "King" and "Queen" would be near each other, indicating their similarity and relation.
- This proximity allows us to understand semantic relationships – words with similar vector values are similar in meaning

How are Word Vectors Created?

- Word2Vec utilizes neural networks to learn these word embeddings.
- There are two main architectures (or methods) to achieve this:
 - **Continuous Bag of Words (CBOW)**
 - **Skip-gram**

Continuous Bag of Words (CBOW)

- Approach: Given a set of context words, CBOW aims to predict the target (center) word.
- Example Sentence: "This is my favourite book"
- Context Window: A "window" of words around the target word is used. For instance, a window of 5 means considering 2 words before and 2 words after the target word

Continuous Bag of Words (CBOW)

Process (Neural Network)

- **Input Layer:** Context words are converted into numerical representations (e.g., using a One-Hot Encoding-like structure)
- **Hidden Layer:** These numerical inputs are fed into a neural network's hidden layer. Connections are formed (e.g., 6x5 matrix connections between neurons)
- **Softmax Layer/Output Layer:** The hidden layer's output is processed to predict the target word (e.g., "my")
- **Training:** The model is trained to minimize the prediction error (loss). As it learns to accurately predict the target word from its context, the weights in the hidden layer become the word vectors

Continuous Bag of Words (CBOW)

- **Vector Extraction:** The numerical values (features) learned in the hidden layer for each word become its vector representation. The dimension of these vectors is typically determined by the **vector_size** or the context window setting

Skip-gram

- **Approach:** Skip-gram is the opposite of CBOW
- Given a target (center) word, Skip-gram aims to predict its surrounding context words
- **Process:** Similar neural network architecture, but the input and output roles are reversed compared to CBOW

Advantages of Word2Vec

- **Captures Semantic Relationship:** The primary advantage is its ability to understand and represent the meaning and relationships between words in a continuous vector space
- **Reduced Dimensionality:** Unlike One-Hot Encoding, BoW, or TF-IDF, Word2Vec can represent words in lower-dimensional dense vectors, making them more efficient for processing.
- **Generalization:** The learned embeddings can generalize well to unseen contexts due to the semantic understanding

Conclusion

- Word2Vec successfully converts words into vectors
- These vectors are not just numerical values; they inherently capture the semantic relationships between words
- This breakthrough helps solve the long-standing problem of understanding context and meaning in natural language processing

